# 1143. Longest Common Subsequence

ory: 2-D Dynamic Programming | LeetCode: https://leetcode.com/problems/longe

# Problem

- Return the length of the longest subsequence common to text1 and text2.

# Intuition

- Dp[i][j] = LCS of the first i chars of a and the first j chars of b.
- If a[i-1] == b[j-1], extend: dp[i-1][j-1] + 1. Otherwise drop one char: max(dp[i-1][j], dp[i][j-1]).

# Brute Force

- Enumerate subsequences of one string and test against the other: O(2^n  m).

# Optimal Approach

- (m+1) x (n+1) table of zeros.
- Fill row by row using the rule above. Answer dp[m][n].
- Can be compressed to two rows.

# Worked Example

```
a = "abcde", b = "ace".

      ""  a  c  e
""     0  0  0  0
a      0  1  1  1
b      0  1  1  1
c      0  1  2  2
d      0  1  2  2
e      0  1  2  3   -> 3 ("ace")
```

# Java Solution

```java
public class LongestCommonSubsequence {
    public int longestCommonSubsequence(String a, String b) {
        int m = a.length(), n = b.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++)
            for (int j = 1; j <= n; j++)
                dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1) ? dp[i - 1][j - 1] + 1 : Math.max(dp[i - 1][j], dp[i][j - 1]);
        return dp[m][n];
    }
}
```

# Complexity

- Time: O(m  n). Space: O(m  n), or O(min(m, n)) compressed.

# Pitfalls

- Off-by-one: table indices are lengths, string indices are i-1 / j-1.